FIRE BRIGADE

Universidad Distrital Francisco José de Caldas

School of Engineering

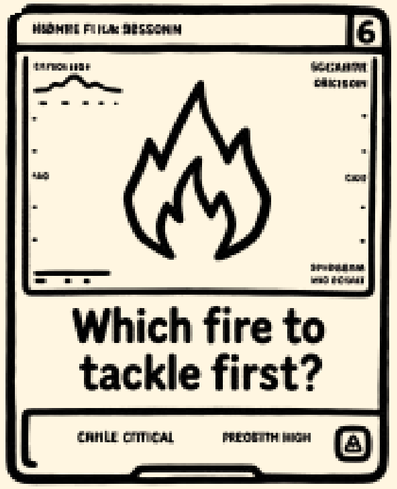
A. Mauricio Cepeda Villanueva - 20242020010

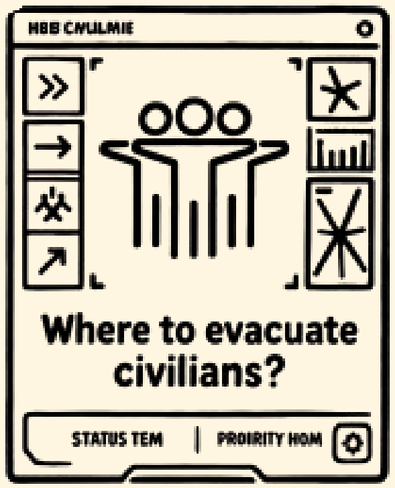
Raúl Andrés Diaz Lozada - 20232020058

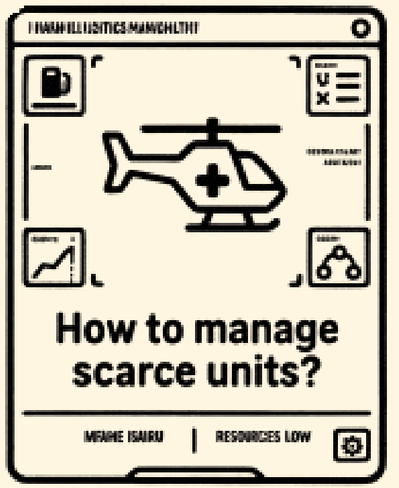
Jeison Felipe Cuervo Huertas - 20251020014

GENERAL INTRODUCTION

This project, Fire Brigade, addresses forest fire emergencies where, under pressure, questions arise such as:



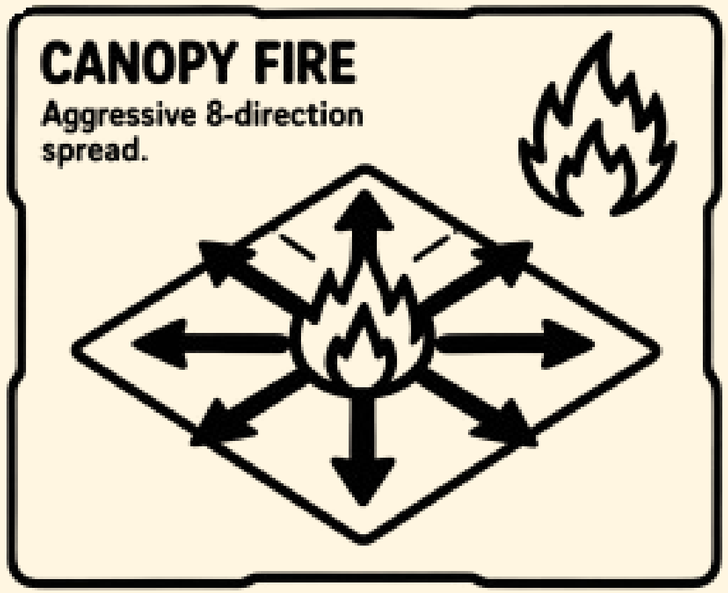




This is where **FireBrigade** comes in

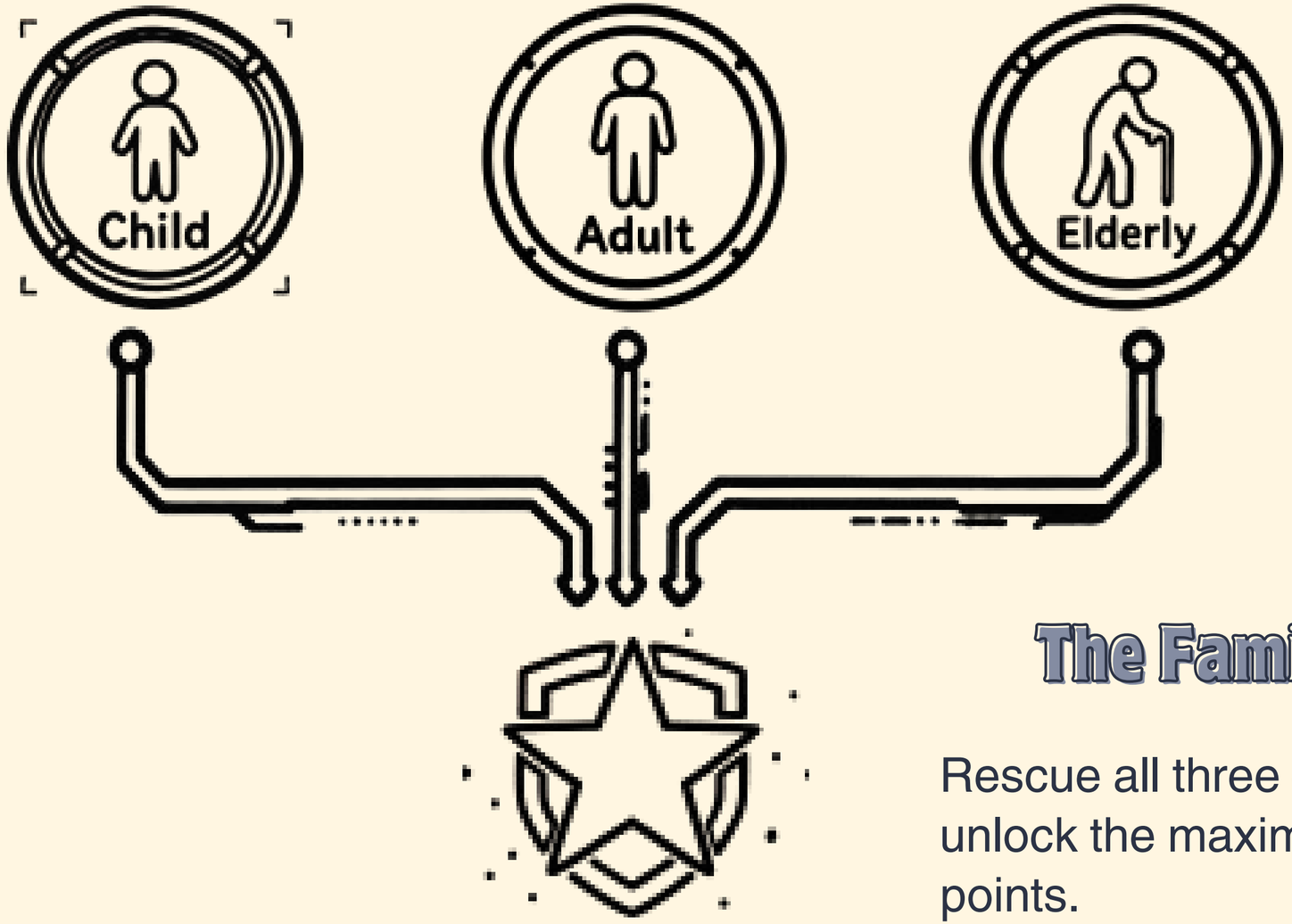
What is it?

Fire Brigade is a turn-based tactical simulator.
The scenario is an 8x8 grid demands spatial strategy against two distinct fire variants, where each cell represents a forest area.



It's not an action game, but rather one of **strategy** and **algorithmic decision-making.**

Evacuation Targets



The Family Bonus

Rescue all three civilian types to unlock the maximum strategic points.

The Role of the Player

The player takes on the role of a fire commander. The player doesn't decide where to send the units or the evacuation route. **That's determined by the algorithms**.

Greedy Algorithm
Unit Deployment logic.

Backtracking
Evacuation Route logic.

The player defines the strategy; the algorithms execute the tactics.

THE CENTRAL PROPOSAL

Demonstrate how two classic algorithms can be integrated into an interactive game.

The first is the Greedy algorithm, which always sends units to the biggest fire on the map.

The second is the Backtracking algorithm, which looks for the safest route to evacuate civilians, avoiding burning cells.

Both algorithms are mandatory and are always present in every game.

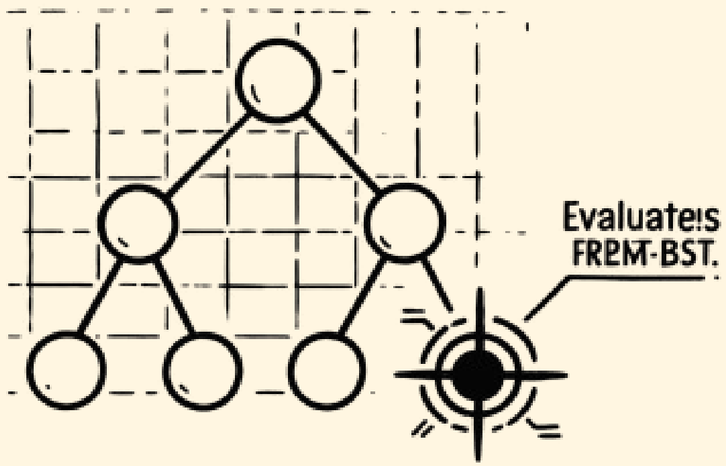
In FireBrigade

There are two mandatory algorithms that determine the tactical actions of the game.

ALGORITHM 1

GREEDY

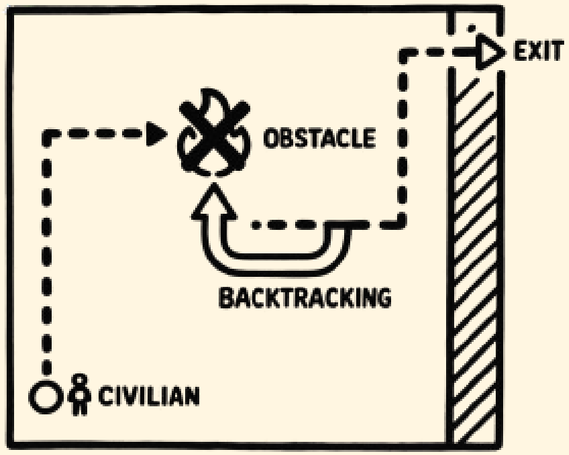
It decides where to deploy each unit.



ALGORITHM 2

BACKTRAKING

Find evacuation routes for civilians



Game variations

The game incorporates three variants that add strategic depth.

First, scarce resources: not all units are available from the start, but arrive at different times depending on the difficulty level. Second, civilians with priorities: there are four types—child, adult, elderly, and family group—each with a point value, and the elderly move the slowest. Third, dynamic fire: there are two types—normal fire that spreads in 4 directions, and cup fire that spreads in 8 directions, making it more aggressive.

Game variations

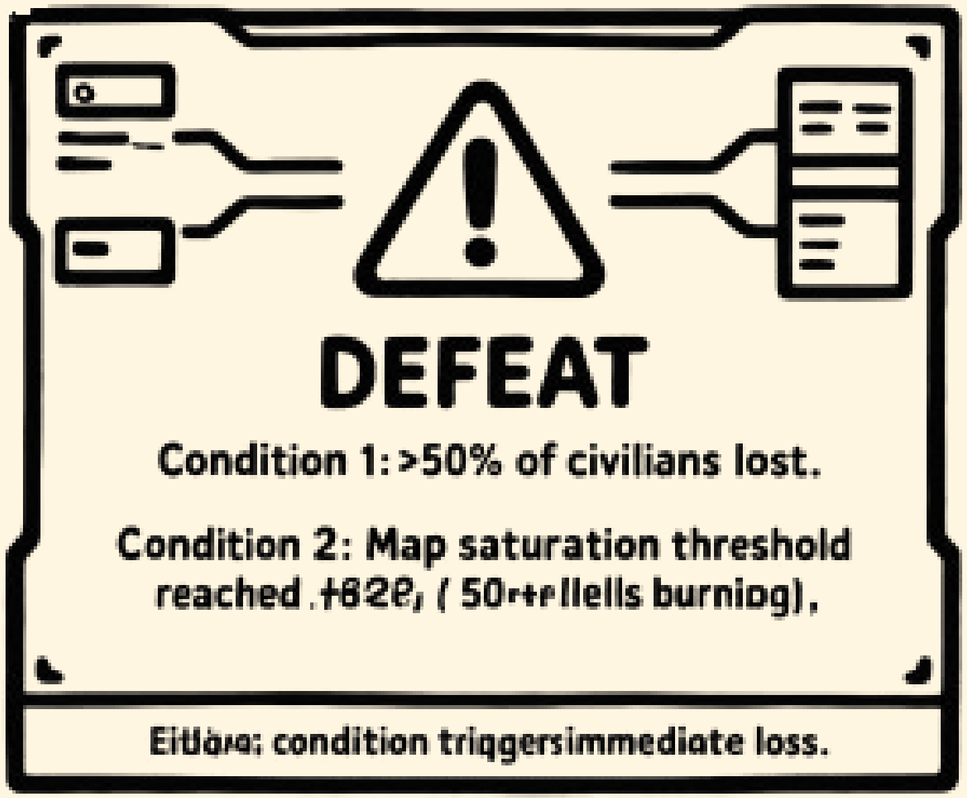
In addition, there are three difficulty levels: Easy, Medium, and Hard. Each one modifies the number and type of fires, the composition of civilians, the starting units, and the frequency with which new units arrive. On the Hard level, for example, there are more cup fires, more civilians, and longer wait times, requiring more careful resource management.

WIN CONDITIONS

The player wins the game when both of the following conditions are satisfied simultaneously

LOSS CONDITIONS

The player loses the game if either of the following conditions occurs:



TURN STRUCTURE

01. Fire Propagation

The C++ engine expands each active fire cell to its neighbours according
to its type:

Normal Fire:

Propagates to the four cardinal directions (up, down, left, right).

Canopy fire:

Propagates to all eight neighbouring cells (including diagonals).

02. Unit arrival:

Any units scheduled to arrive on the current turn are appended to the tail of the linked list queue. Arrival intervals are defined by the difficulty level.

TURN STRUCTURE

03. Player actions:

The player may perform one or more actions while units remain available in
the queue or civilians remain on the map

Deploy a unit

The player selects a unit type from the queue

Evacuate a civilian

The player selects a civilian on the grid; the Backtracking algorithm computes the shortest safe route to the border

04. State update

After player actions are completed, the engine updates the BST with any changes

SYSTEM INTEGRATION

state.json
(Full map state)

C++ ENGINE
Manages game state, fires, and grid propogation.

PYTHON
Decision-making (Greedy & Backtracking).

input.json
(Chosen tactical actions)

JSON COMMUNICATION

The system uses a JSON-based communication between C++ and Python.

- state.json: Generated by the C++ engine. Contains the full game state (fires, units, civilians, risk zones).
- input.json: Generated by Python. Contains only the selected action (DEPLOY or EVACUATE) and its parameters.

This approach ensures a clear separation of responsibilities:
C++ manages the state, while Python handles decision-making using Greedy and Backtracking algorithms.

PSEUDOCODE

```
ALGORITHM GreedyDispatcher
INPUT:
   - bst: Binary Search Tree with burning cells ordered by risk
   - unit_type: type of unit to deploy (CREW, TRUCK, HELICOPTER)
OUTPUT:
   - target_cell: coordinates (row, column)

BEGIN
   // Get the cell with the highest risk from the BST
   target_cell ← bst.getHighestRiskCell()

   // Handle ties when multiple cells have the same risk
   IF multiple cells have the same risk value THEN
      // Priority 1: Prefer canopy fires (more aggressive)
      IF any canopy fire exists among tied cells THEN
         target_cell ← select cell with canopy fire
      ELSE
         // Priority 2: Select by smallest coordinates
         target_cell ← select cell with smallest row, then smallest column
      END IF
   END IF

   RETURN target_cell
                                                    END
```

In summary:
Get the cell with the highest risk from the BST. If there's a tie:

- Prioritize the highest risk cell
- If still a tie, choose the smallest coordinates

and return the destination cell

PSEUDOCODE

```
ALGORITHM BacktrackingEvacuation
INPUT:
   - start: coordinates (row, column) of the civilian
   - obstacles: 8×8 matrix where True = burning cell (blocked)
   - civilian_type: CHILD, ADULT, ELDERLY, FAMILY_GROUP
OUTPUT:
   - route: list of coordinates [(r1,c1), (r2,c2), ..., (exit)]
   - success: boolean

BEGIN
   visited ← 8×8 matrix initialized to False
   current_route ← empty list
   best_route ← empty list
   best_length ← INFINITY

   success ← explore(start, visited, current_route)

   IF success THEN
      RETURN best_route, True
   ELSE
      RETURN empty list, False
   END IF
                                                    END
```

PSEUDOCODE

```
FUNCTION explore(position, visited, current_route)
  Append position to current_route

  // Base case: reached the border (exit condition)
  IF position.row == 0 OR position.row == 7 OR
    position.col == 0 OR position.col == 7 THEN

    IF length(current_route) < best_length THEN
      best_route ← copy(current_route)
      best_length ← length(current_route)
    END IF
    RETURN True
  END IF

  // Mark current cell as visited
  visited[position.row][position.col] ← True

  // Define possible movements: up, down, left, right
  movements ← [(-1,0), (1,0), (0,-1), (0,1)]

  FOR EACH movement IN movements DO
    new_row ← position.row + movement.row
    new_col ← position.col + movement.col

    IF isWithinBounds(new_row, new_col) AND
      NOT obstacles[new_row][new_col] AND
      NOT visited[new_row][new_col] THEN

      explore((new_row, new_col), visited, current_route)
    END IF
  END FOR

  // Backtrack: unmark current cell and remove from route
  visited[position.row][position.col] ← False
  Remove last element from current_route

  RETURN False
END

FUNCTION isWithinBounds(row, column)
  RETURN row >= 0 AND row < 8 AND column >= 0 AND column < 8
                                                    END
```

In summary:
From the civilian's position, explore 4 directions
> Mark visited cells to avoid cycles
> If the edge is reached, record the path
> If there is no way out, backtrack and try another path
> Return the shortest path.

¡Thanks!